

Proxy, Observer & Adapter Patterns in Java Spring Boot Microservices

Proxy, Observer & Adapter Patterns in Java Spring Boot Microservices (0-7 Years Guide)

PART 1 — PROXY PATTERN

1. What It Is

Proxy provides a **surrogate/placeholder object** that controls access to another object, letting you add behavior (security, caching, transactions, logging) **around** a method call without changing the original class's code.

Why it matters: cross-cutting concerns (things needed everywhere — transactions, caching, security, retries) get applied declaratively via annotations instead of being hand-written in every method.

2. Classic Java Implementation (Without Spring)

```
public interface UserService {
    User getUser(Long id);
}

public class UserServiceImpl implements UserService {
    public User getUser(Long id) { return new User(id, "Raj"); }
}

// Manual proxy adding logging around the real object
public class LoggingUserServiceProxy implements UserService {
    private final UserService target;

    public LoggingUserServiceProxy(UserService target) { this.target
= target; }

    public User getUser(Long id) {
        System.out.println("Before call: getUser(" + id + ")");
        User result = target.getUser(id);
        System.out.println("After call: " + result);
        return result;
    }
}
```

The caller talks to LoggingUserServiceProxy, unaware that it's wrapping the real implementation.

3. How Spring AOP Uses This Internally

When you annotate a bean method with `@Transactional`, `@Cacheable`, `@Async`, or `@Retryable`, Spring **does not modify your class**. Instead, at startup it wraps your bean in a **dynamic proxy** (JDK dynamic proxy if your class implements an interface, or a CGLIB subclass proxy if it doesn't). Every call to that bean actually goes through the proxy first.

```
@Service
public class OrderService {

    @Transactional
    public void placeOrder(Order order) {
        orderRepository.save(order);
        inventoryService.reserve(order);
    }
}
```

What actually happens at runtime: 1. Spring injects a **proxy**, not your raw `OrderService` instance, wherever `OrderService` is autowired 2. Calling `placeOrder()` on the proxy: proxy starts a transaction → calls your real method → commits on success / rolls back on exception 3. Your class has **zero transaction-handling code** — the proxy supplies it

4. @Transactional Internals

```
@Service
public class PaymentService {
    @Transactional
    public void processPayment(Payment payment) {
        paymentRepository.save(payment);
        ledgerService.recordEntry(payment); // if this throws, the
save() above rolls back too
    }
}
```

The proxy wraps the method call in `TransactionInterceptor` logic: begin transaction → invoke your method → commit if no exception → rollback if a `RuntimeException/Error` (unchecked, by default) propagates out.

Famous gotcha (frequently asked): self-invocation bypasses the proxy.

```
@Service
public class OrderService {
    public void placeOrder(Order order) {
        this.saveWithTransaction(order); // BYPASSES the proxy –
@Transactional has NO EFFECT here
    }

    @Transactional
    public void saveWithTransaction(Order order) { ... }
}
```

Because `this.saveWithTransaction()` calls the method directly on the real object, not through the Spring-managed proxy, so the transactional advice never triggers.

5. @Cacheable Internals

```
@Service
public class ProductService {
    @Cacheable(value = "products", key = "#id")
    public Product getProduct(Long id) {
        return productRepository.findById(id).orElseThrow(); // only
runs on cache miss
    }
}
```

```
}
```

The proxy intercepts the call: checks the cache (Redis/Caffeine/etc.) for the key first — if present, returns the cached value **without ever invoking your method body**; if absent, calls your method, then stores the result under that key before returning it.

6. @Async Internals

```
@Service
public class NotificationService {
    @Async
    public void sendEmail(String to, String message) {
        // runs on a separate thread from a task executor, not the
        // caller's thread
    }
}
```

The proxy submits your method call to a `TaskExecutor` thread pool and returns immediately (or returns a `CompletableFuture` if your method declares one), instead of running it synchronously on the calling thread.

7. JDK Dynamic Proxy vs CGLIB (Frequently Asked)

	JDK Dynamic Proxy	CGLIB Proxy
Requires	Class implements at least one interface	Works on concrete classes too
How	Implements the same interface(s) at runtime	Generates a runtime subclass
Spring default	Used when interfaces are present	Used automatically as fallback for interface-less classes (default in Spring Boot since Spring 4)
Limitation	Only proxies interface methods	Can't proxy final classes/methods

8. Where Proxy Is Used in Microservices

- `@Transactional` boundaries around service methods calling repositories
- `@Cacheable/@CacheEvict/@CachePut` around expensive/frequently-read lookups
- `@Async` for fire-and-forget operations (sending notifications, audit logging)
- `@Retryable` (Resilience4j/Spring Retry) wrapping calls to unreliable downstream services
- `@PreAuthorize/@Secured` — security proxies enforcing method-level access control
- Feign clients themselves are proxies — you write an interface, Spring Cloud OpenFeign generates the actual HTTP-calling implementation at runtime

```
@FeignClient(name = "inventory-service")
public interface InventoryClient {
    @GetMapping("/stock/{sku}")
    StockResponse getStock(@PathVariable String sku);
    // no implementation - Feign generates a dynamic proxy that
    // makes the HTTP call
}
```

}

9. Proxy — By Experience Level

0-2 Years

Expected: Know that `@Transactional/@Cacheable/@Async` “just work” via annotations; basic idea that Spring wraps beans. **Common**

questions: What does `@Transactional` do? What is `@Cacheable` used for?

2-4 Years

Expected: Understand the self-invocation gotcha; difference between JDK dynamic proxy and CGLIB; debug a case where

`@Transactional/@Async` silently doesn’t work. **Common questions:** Why doesn’t `@Transactional` work when a method calls another method in the same class? What’s the difference between JDK proxy and CGLIB proxy, and when does Spring use each?

4-7 Years

Expected: Explain full AOP proxy creation lifecycle (`BeanPostProcessor` wrapping beans post-instantiation); design custom `@Aspect/@Around` advice for cross-cutting concerns (audit logging, metrics, rate limiting) instead of relying only on built-in annotations; understand performance overhead of proxying at scale; know how to combine multiple proxies on one bean (`@Transactional + @Cacheable + @Retryable`) and their execution order. **Common questions:** Walk through what happens internally from bean instantiation to a fully-proxied bean being returned by the container. Design a custom `@Aspect` to automatically log execution time of every service method. What’s the ordering behavior when a method has both `@Transactional` and `@Cacheable`, and why does it matter? How would you apply distributed tracing/correlation IDs across service calls using AOP?

PART 2 — OBSERVER PATTERN

1. What It Is

Observer defines a **one-to-many dependency**: when one object (the subject/publisher) changes state or does something notable, all registered dependents (observers/listeners) are notified automatically — without the subject needing to know their concrete types.

Why it matters: decouples the thing that triggers an event from the things that react to it — you can add new reactions without touching the original triggering code.

2. Classic Java Implementation (Without Spring)

```
public interface OrderObserver {
    void onOrderPlaced(Order order);
}

public class InventoryObserver implements OrderObserver {
    public void onOrderPlaced(Order order) {
        System.out.println("Reserving stock for " + order.getId());
    }
}
```

```

    public class OrderSubject {
        private final List<OrderObserver> observers = new ArrayList<>();

        public void subscribe(OrderObserver observer) {
observers.add(observer); }

        public void placeOrder(Order order) {
// ... save order ...
observers.forEach(o -> o.onOrderPlaced(order)); // notify
all
        }
    }
}

```

3. Spring's Built-In Implementation — ApplicationEvent / @EventListener

Spring provides this out of the box via its ApplicationContext's built-in event bus — you don't need to manage an observer list yourself.

Defining an Event

```

public class OrderPlacedEvent extends ApplicationEvent {
    private final Order order;

    public OrderPlacedEvent(Object source, Order order) {
        super(source);
        this.order = order;
    }

    public Order getOrder() { return order; }
}

```

Or, simpler (modern style — POJO event, no inheritance needed):

```

public record OrderPlacedEvent(Order order) { }

```

Publishing the Event

```

@Service
public class OrderService {
    private final ApplicationEventPublisher eventPublisher;

    public OrderService(ApplicationEventPublisher eventPublisher) {
        this.eventPublisher = eventPublisher;
    }

    @Transactional
    public void placeOrder(Order order) {
        orderRepository.save(order);
        eventPublisher.publishEvent(new OrderPlacedEvent(order)); //
notify all listeners
    }
}

```

Listening for the Event

```

@Component
public class InventoryEventListener {
    @EventListener
    public void handleOrderPlaced(OrderPlacedEvent event) {
        System.out.println("Reserving stock for " +
event.order().getId());
    }
}

```

```

@Component
public class NotificationEventListener {
    @EventListener
    public void handleOrderPlaced(OrderPlacedEvent event) {
        System.out.println("Sending confirmation email for " +
event.order().getId());
    }
}

```

OrderService has **no idea** how many listeners exist or what they do — completely decoupled. Adding a third listener (e.g., analytics tracking) requires zero changes to OrderService.

4. Async Event Handling

```

@Component
public class NotificationEventListener {
    @Async
    @EventListener
    public void handleOrderPlaced(OrderPlacedEvent event) {
        // runs on a separate thread – doesn't block the publisher
        (Proxy pattern combined with Observer here)
    }
}

```

(Note: requires @EnableAsync on a configuration class, and the caveats about self-invocation/proxying from Part 1 apply.)

5. Transaction-Bound Events (Important Production Detail)

```

@Component
public class InventoryEventListener {
    @TransactionalEventListener(phase =
TransactionPhase.AFTER_COMMIT)
    public void handleOrderPlaced(OrderPlacedEvent event) {
        // only runs if the surrounding transaction actually commits
        successfully
    }
}

```

This avoids the classic bug where a listener reacts to an event whose originating transaction later rolled back (e.g., reserving inventory for an order that never actually got saved).

6. Where Observer Is Used in Microservices

- **In-process** decoupling: order placed → notify inventory, notification, analytics modules within the same service
- **Audit logging** — listen for domain events and persist an audit trail without cluttering business logic
- **Cache invalidation** — listen for update events to evict stale cache entries
- **Cross-service equivalent:** Kafka/RabbitMQ event publishing is the *distributed* version of this same pattern — a service publishes a domain event, and multiple other *services* (not just in-process listeners) subscribe and react (this is the Choreography-style Saga you covered earlier)

```

@Component
public class OrderEventKafkaPublisher {
    @TransactionalEventListener(phase =

```

```

TransactionPhase.AFTER_COMMIT)
    public void publishToKafka(OrderPlacedEvent event) {
        kafkaTemplate.send("order-events",
            event.order().getId().toString(), event.order());
        // bridges the in-process Observer event to a distributed,
        // cross-service Observer (Kafka topic subscribers)
    }
}

```

7. Observer — By Experience Level

0-2 Years

Expected: Know what `@EventListener` does, publish and consume a simple event within one service. **Common questions:** What is the Observer pattern? How do you publish and listen for an event in Spring?

2-4 Years

Expected: Use `@Async` with `@EventListener` for non-blocking side effects; use `@TransactionalEventListener` to avoid acting on uncommitted data; explain why Observer decouples publisher from subscriber logic. **Common questions:** What's the risk of using a plain `@EventListener` inside a `@Transactional` method, and how does `@TransactionalEventListener` fix it? How would you send a notification email without slowing down the main request thread?

4-7 Years

Expected: Bridge in-process Observer (Spring events) to distributed Observer (Kafka/RabbitMQ) correctly, including transactional-outbox considerations to avoid dual-write problems; design event schemas that multiple listeners/services can consume without tight coupling; handle event ordering, idempotency, and failure/retry of listeners at scale; know the limits of Spring's in-process event system (single JVM only — doesn't survive a pod crash or scale across replicas without an external broker). **Common questions:** How would you reliably publish a domain event to Kafka only if the database transaction commits (avoiding the dual-write problem)? What happens to in-process Spring events if your pod crashes mid-processing — how is that different from a Kafka-based Observer? How do you handle event listener failures without blocking or losing the original event (retry topics, dead-letter queues)?

PART 3 — ADAPTER PATTERN

1. What It Is

Adapter converts the interface of a class into another interface that clients expect, letting incompatible interfaces work together — typically used to wrap a third-party library or legacy system behind an interface your application controls.

Why it matters: isolates your codebase from external API changes; lets you swap vendors/legacy systems without rewriting business logic; makes third-party dependencies mockable/testable.

2. Classic Problem It Solves

```

// Third-party SDK – you don't control this interface, and it
doesn't match what your app expects
public class LegacyPaymentGatewaySDK {
    public String makePayment(String accountNo, String amountStr,
String currencyCode) {
        return "SUCCESS"; // returns a raw string, awkward types
    }
}

// Your application's own clean interface
public interface PaymentGateway {
    PaymentResult pay(PaymentRequest request);
}

// Adapter bridges the two
@Component
public class LegacyPaymentGatewayAdapter implements PaymentGateway {
    private final LegacyPaymentGatewaySDK legacySdk;

    public LegacyPaymentGatewayAdapter(LegacyPaymentGatewaySDK
legacySdk) {
        this.legacySdk = legacySdk;
    }

    @Override
    public PaymentResult pay(PaymentRequest request) {
        String result = legacySdk.makePayment(
            request.getAccountNumber(),
            String.valueOf(request.getAmount()),
            request.getCurrency().getCode()
        );
        return "SUCCESS".equals(result) ? PaymentResult.success() :
PaymentResult.failure();
    }
}

```

Your `OrderService` depends only on `PaymentGateway` — it never knows the legacy SDK exists. If you later switch vendors, you write a new adapter; `OrderService` doesn't change at all.

3. Real Example — Wrapping a Third-Party Notification SDK

```

// Third-party library with its own vendor-specific API
public class TwilioSmsClient {
    public void sendSms(String fromNumber, String toNumber, String
body) { ... }
}

public interface SmsGateway {
    void send(String to, String message);
}

@Component
public class TwilioSmsAdapter implements SmsGateway {
    private final TwilioSmsClient twilioClient;
    private final String fromNumber;

    public TwilioSmsAdapter(TwilioSmsClient twilioClient,
@Value("${sms.from-number}") String fromNumber) {
        this.twilioClient = twilioClient;
        this.fromNumber = fromNumber;
    }

    @Override
    public void send(String to, String message) {
        twilioClient.sendSms(fromNumber, to, message);
    }
}

```

```
}  
}
```

If you later migrate from Twilio to AWS SNS, you write `SnsAdapter` implements `SmsGateway` — nothing else in the codebase changes (this combines nicely with the Strategy/Factory pattern if you need both providers active simultaneously).

4. Adapter vs Facade vs Decorator (Common Interview Confusion)

Pattern	Purpose
Adapter	Makes an <i>incompatible</i> interface usable by <i>converting</i> it to the one you need — the shape changes
Facade	Provides a <i>simplified</i> interface over a <i>complex</i> subsystem — hides complexity, doesn't necessarily change the interface shape
Decorator	<i>Adds new behavior</i> on top of an object while keeping the <i>same</i> interface — enhances, doesn't convert

5. Where Spring Itself Uses Adapter Internally

- `HandlerAdapter` — Spring MVC's `DispatcherServlet` uses adapters to invoke different types of controller handlers (annotated controllers, functional endpoints, etc.) through one uniform mechanism
- `JpaVendorAdapter` — adapts vendor-specific JPA provider quirks (Hibernate, EclipseLink) to a common Spring JPA configuration interface
- Spring's `Slf4jLogger` / logging abstraction — adapts multiple logging frameworks (Logback, Log4j2) behind a single `Logger` interface

6. Where Adapter Is Used in Microservices

- Wrapping third-party payment gateways, SMS/email providers, cloud storage SDKs (S3, Azure Blob) behind your own interface
- Wrapping legacy SOAP services behind a modern REST-friendly interface your app consumes
- Adapting a legacy monolith's data format to the schema your new microservice expects, during a Strangler Fig migration
- Wrapping a cloud provider's SDK (AWS/GCP/Azure) so your business logic isn't locked into one vendor's API shape

```
public interface FileStorage {  
    String upload(String key, byte[] content);  
}  
  
@Component  
@ConditionalOnProperty(name = "storage.provider", havingValue =  
"s3")  
public class S3StorageAdapter implements FileStorage {  
    private final S3Client s3Client; // AWS SDK client – vendor-  
specific  
    public String upload(String key, byte[] content) {
```

```

        s3Client.putObject(...);
        return key;
    }
}

```

7. Adapter — By Experience Level

0-2 Years

Expected: Know what Adapter solves (incompatible interfaces), wrap a simple third-party class behind your own interface. **Common questions:** What is the Adapter pattern? Give a real-world analogy (e.g., a power plug adapter).

2-4 Years

Expected: Wrap an actual third-party SDK (payment/SMS/storage) behind your own domain interface; explain Adapter vs Facade vs Decorator; make third-party dependencies mockable in unit tests via the adapter interface. **Common questions:** How would you integrate a third-party payment SDK without letting its API leak into your business logic? Difference between Adapter and Facade? How does wrapping a dependency in an Adapter help with unit testing?

4-7 Years

Expected: Design vendor-agnostic abstractions across multiple adapters selected via configuration (@ConditionalOnProperty) or Strategy/Factory; use Adapter as an anti-corruption layer during a Strangler Fig migration from a legacy monolith; handle differing error models/retry semantics across multiple third-party adapters uniformly; evaluate the cost/benefit of adapting vs directly depending on a vendor SDK for a small, unlikely-to-change integration. **Common questions:** How would you design your system so switching cloud storage providers (S3 → GCS) requires touching only one class? What's an "anti-corruption layer" and how does Adapter implement that concept during a monolith-to-microservices migration? How do you normalize error handling when three different third-party SDKs each throw completely different exception types?

Quick Summary Table

Pattern	Core Purpose	Key Spring Example
Proxy	Add behavior around a method call transparently	@Transactional, @Cacheable, @Async, Feign clients
Observer	One-to-many notification without tight coupling	ApplicationEvent, @EventListener, @TransactionalEventListener
Adapter	Bridge an incompatible interface to the one you need	Wrapping payment/SMS/cloud SDKs behind your own interface

How They Often Combine (Senior-Level Integration Point)

“An order is placed. It should reserve inventory, send a confirmation SMS via a third-party provider, and publish an event to Kafka — but only if the database transaction commits, and none of this should block the HTTP response.”

- **Observer** (@TransactionalEventListener, AFTER_COMMIT) ensures reactions only fire after a successful commit
- **Proxy** (@Async) ensures the SMS-sending listener runs off the main thread so the caller isn't blocked
- **Adapter** (SmsGateway wrapping the vendor SDK) isolates the business logic from the SMS provider's specific API

```
@Component
public class OrderConfirmationListener {
    private final SmsGateway smsGateway; // Adapter

    @Async // Proxy
    @TransactionalEventListener(phase = AFTER_COMMIT) // Observer
    public void onOrderPlaced(OrderPlacedEvent event) {
        smsGateway.send(event.order().getCustomerPhone(), "Order
confirmed!");
    }
}
```